

assignment_3

Saba Naji

Part 1

```
#install.packages("imager")  
library(imager)
```

```
## Warning: package 'imager' was built under R version 4.5.2
```

```
## Loading required package: magrittr
```

```
##  
## Attaching package: 'imager'
```

```
## The following object is masked from 'package:magrittr':  
##  
##   add
```

```
## The following objects are masked from 'package:stats':  
##  
##   convolve, spectrum
```

```
## The following object is masked from 'package:graphics':  
##  
##   frame
```

```
## The following object is masked from 'package:base':  
##  
##   save.image
```

```
image_dir <- "data"  
image_height <- 64  
image_width <- 64
```

```
image_files <- list.files(  
  image_dir,  
  pattern = "\\.(jpg|jpeg|png|bmp)$",  
  ignore.case = TRUE,  
  full.names = TRUE  
)  
  
image_num <- length(image_files)  
  
read_to_vector <- function(path, H = image_height, W = image_width){  
  im <- load.image(path)  
  im <- grayscale(im)  
  im <- imager::resize(im, size_x = W, size_y = H)  
  as.numeric(im)  
}  
  
image_matrix_list <- lapply(image_files, read_to_vector)  
x <- do.call(rbind, image_matrix_list)  
dim(x)
```

```
## [1] 124 4096
```

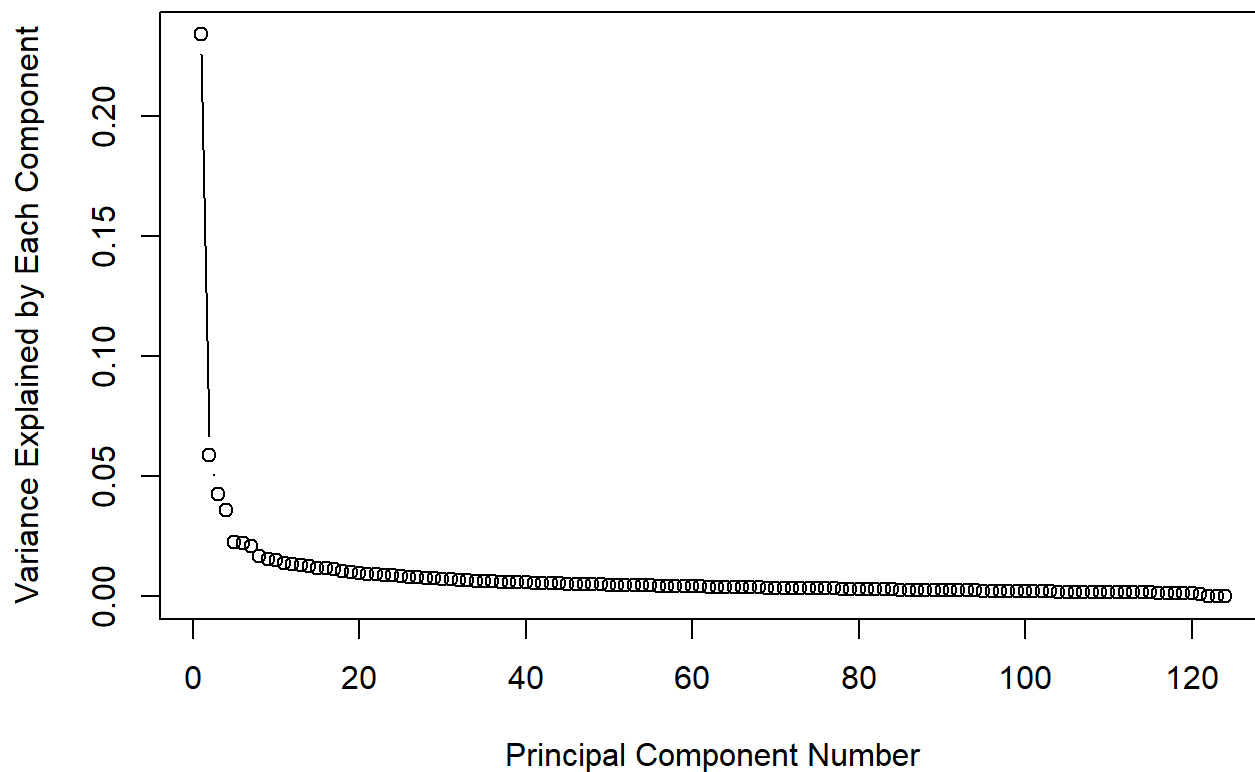
This code loads all images, converts them to grayscale, resizes them to 64×64, flattens them, and stacks them into a matrix with 124 images and 4096 pixels per image.

```
pca_full <- prcomp(x,  
  center = TRUE,  
  scale. = FALSE)  
var_explained <- pca_full$sdev^2  
var_ratio <- var_explained / sum(var_explained)  
cumvar <- cumsum(var_ratio)
```

This code performs a full PCA on the image matrix X. The data are centered but not scaled, which keeps the original pixel intensity ranges. After PCA is computed, the code extracts the variance of each principal component by squaring their standard deviation. It then converts these values into variance ratios, showing how much of the total data variance each component explains. Finally, it computes the cumulative variance, which indicates how many components are needed to retain a chosen percentage of the information in the original images.

```
plot(  
  var_ratio,  
  type = "b",  
  xlab = "Principal Component Number",  
  ylab = "Variance Explained by Each Component",  
  main = "Scree Plot - PCA")
```

Scree Plot - PCA



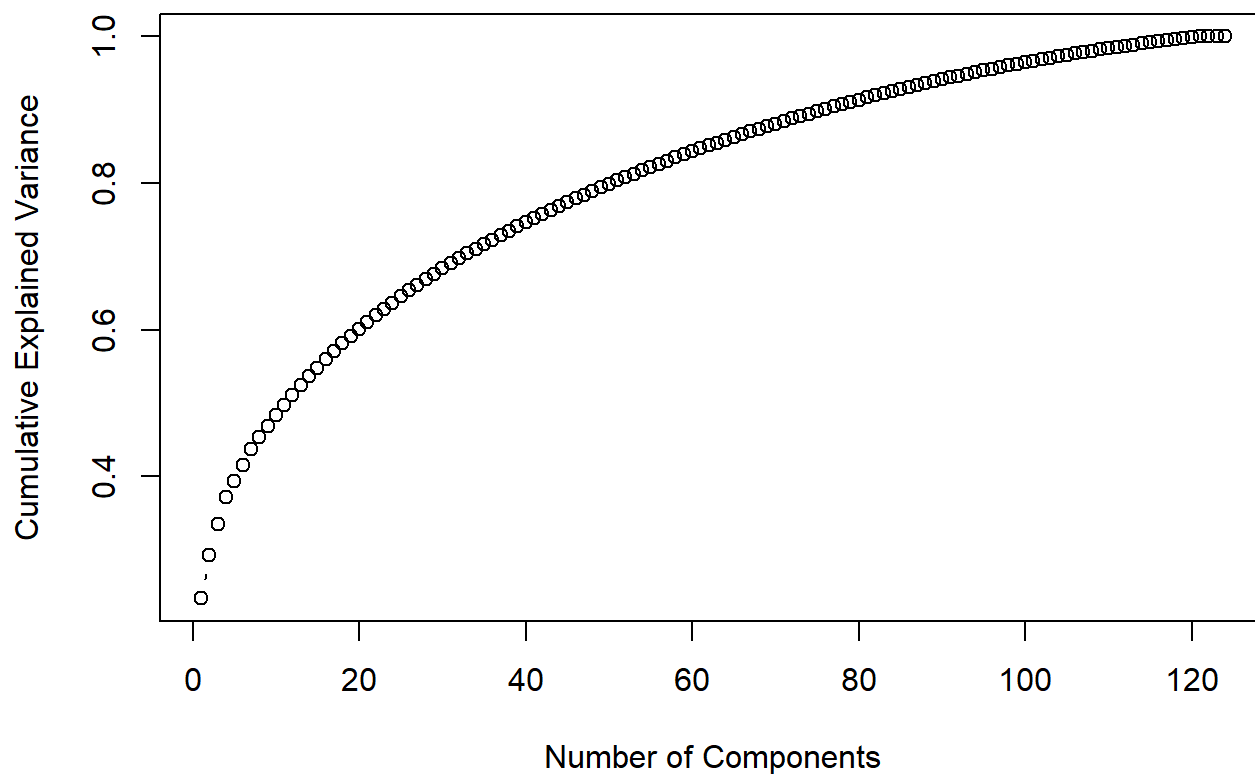
The

scree plot displays the amount of variance explained by each principal component (PC) in the PCA model. The curve shows a sharp decline after the first few components, indicating that PC1 and a small number of early components capture most of the meaningful variance in the images. After approximately the 5th–10th component, the plot becomes almost flat, meaning that additional components contribute very little new information.

Overall, the plot demonstrates that the dataset has a strong low-dimensional structure and that a small number of principal components is sufficient for representing the essential information, while the remaining components mostly reflect noise.

```
plot(  
  cumvar,  
  type = "b",  
  xlab = "Number of Components",  
  ylab = "Cumulative Explained Variance",  
  main = "Cumulative Explained Variance"  
)
```

Cumulative Explained Variance



cumulative explained variance curve shows that the first principal components capture most of the information in the dataset. The curve rises steeply for the first 20–30 components, indicating that these components contain the dominant structure of the images. After roughly 50 components, the curve becomes much flatter, meaning that additional components contribute little new information and mostly represent noise or fine-grained variations. Based on this behavior, selecting a value of k in the range of 30–50 provides an effective balance between dimensionality reduction and information retention.

```
which(cumvar >= 0.90)[1]
```

```
## [1] 76
```

I chose $k = 76$ principal components. The cumulative explained variance plot shows that with 76 components, the PCA model reaches about 90% of the total variance. After this point, the curve becomes almost flat, meaning additional components add very little new information. This makes 76 a good cutoff: it keeps most of the important structure in the images while avoiding unnecessary extra components.

```
k_medium <- 76  
k_small <- 20  
k_large <- 110
```

```

reconstruct_with_k <- function(pca_obj, k){
  scores_k <- pca_obj$x[, 1:k]

  rotation_k <- pca_obj$rotation[, 1:k]
  x_hat_centered <- scores_k %*% t(rotation_k)
  x_hat <- sweep(x_hat_centered, 2, pca_obj$center, "+")

  x_hat
}

x_small <- reconstruct_with_k(pca_full, k_small)
x_medium <- reconstruct_with_k(pca_full, k_medium)
x_large <- reconstruct_with_k(pca_full, k_large)

```

The function `reconstruct_with_k()` rebuilds the images using only the first k PCA components. It takes the reduced PCA scores and loadings, reconstructs an approximation of the original data, and then adds back the mean of the images. This gives a version of each image using only limited PCA information.

```

show_image <- function(vec, title = "") {
  im <- as.cimg(vec, x = image_width, y = image_height, cc = 1)
  plot(im, axes = FALSE, main = title)
}

idx <- c(1, 10, 50)

par(mfrow = c(length(idx), 4))

for (i in idx) {
  show_image(x[i, ], title = "Original")
  show_image(x_small[i, ], title = paste("k =", k_small))
  show_image(x_medium[i, ], title = paste("k =", k_medium))
  show_image(x_large[i, ], title = paste("k =", k_large))
}

```

Original**k = 20****k = 76****k = 110****Original****k = 20****k = 76****k = 110****Original****k = 20****k = 76****k = 110**

The code compares each original image with reconstructions made using $k = 20$, $k = 76$, and $k = 110$ PCA components. - With 20 components, the images are very blurry and lose most details. - With 76 components, the main structures and features are clear and much closer to the original. - With 110 components, the images are almost identical to the originals.

Part 2

```
file_names <- basename(image_files)

labels <- ifelse(
  grepl("^c",file_names, ignore.case = TRUE),
  "Chelsea",
  "ManUtd")

labels <- factor(labels)
table(labels)
```

```
## labels
## Chelsea  ManUtd
##      65      59
```

The output shows that the dataset contains 65 Chelsea images and 59 Manchester United images, and these labels are used in all subsequent training and evaluation procedures.

```
set.seed(123)

k_feat <- k_medium
Z_all <- pca_full$x[, 1:k_feat]

dim(Z_all)
```

```
## [1] 124 76
```

The code extracts the first 76 PCA features for all images. The output 124 76 means the dataset now contains 124 images, each represented by 76 reduced features instead of the original 4096 pixels. This compact feature set will be used for classification.

```
set.seed(123)
train_idx <- sample(nrow(Z_all), 0.8 * nrow(Z_all))

X_train <- Z_all[train_idx, ]
X_test  <- Z_all[-train_idx, ]
y_train <- labels[train_idx]
y_test  <- labels[-train_idx]
```

This code randomly selects 80% of the data for training and keeps the remaining 20% for testing. The PCA features (Z_all) are split into X_train and X_test, and the corresponding class labels are split into y_train and y_test. This ensures that the models are trained on one portion of the data and evaluated fairly on unseen samples.

```
metrics_binary <- function(y_true, y_pred, positive = "ManUtd") {  
  y_true <- factor(y_true)  
  y_pred <- factor(y_pred, levels = levels(y_true))  
  
  cm <- table(y_true, y_pred)  
  
  TP <- cm[positive, positive]  
  FP <- sum(cm[, positive]) - TP  
  FN <- sum(cm[positive, ]) - TP  
  TN <- sum(cm) - TP - FP - FN  
  
  precision <- TP / (TP + FP)  
  recall    <- TP / (TP + FN)  
  f1        <- 2 * precision * recall / (precision + recall)  
  accuracy  <- (TP + TN) / sum(cm)  
  
  list(  
    precision = precision,  
    recall    = recall,  
    f1        = f1,  
    accuracy  = accuracy,  
    confusion = cm  
  )  
}
```

This code defines a function that evaluates a binary classification model. It compares the true labels with the predicted labels and calculates four key performance measures: precision, recall, F1-score, and accuracy. It also creates a confusion matrix showing how many predictions were correct or incorrect. The function treats "ManUtd" as the positive class when computing these metrics.


```

library(class)

Kfold <- 5
k_grid <- c(1, 3, 5, 7, 9, 11)

set.seed(123)
fold_id <- sample(rep(1:Kfold, length.out = length(y_train)))

cv_f1_knn <- sapply(k_grid, function(k) {
  f1_per_fold <- sapply(1:Kfold, function(fold) {
    val <- fold_id == fold
    tr <- !val

    pred_val <- knn(
      train = X_train[tr, , drop = FALSE],
      test  = X_train[val, , drop = FALSE],
      cl    = y_train[tr],
      k     = k
    )

    metrics_binary(y_train[val], pred_val, positive = "ManUtd")$f1
  })

  mean(f1_per_fold, na.rm = TRUE)
})

cv_results_knn <- data.frame(
  k = k_grid,
  mean_f1 = cv_f1_knn
)

cv_results_knn

```

k <dbl>	mean_f1 <dbl>
1	0.4866525
3	0.5521242
5	0.6423497
7	0.6572078
9	0.6227090
11	0.5992674

6 rows

```

best_k <- k_grid[which.max(cv_f1_knn)]
best_k

```

```
## [1] 7
```

I used 5-fold cross-validation to choose the best value of k for the KNN classifier. The F1-score was computed for each tested value of k (1, 3, 5, 7, 9, 11).

The highest average F1-score was obtained at $k = 7$, meaning this value gives the best overall performance on the training data. Therefore, $k = 7$ was selected as the optimal number of neighbors.

```
knn_test_pred <- knn(
  train = X_train,
  test  = X_test,
  cl     = y_train,
  k      = best_k
)

metrics_knn_test <- metrics_binary(y_test, knn_test_pred, positive = "ManUtd")

res <- metrics_knn_test

result_table <- data.frame(
  Metric = c("Accuracy", "Precision", "Recall", "F1 Score"),
  Value  = c(res$accuracy, res$precision, res$recall, res$f1)
)

result_table
```

Metric <chr>	Value <dbl>
Accuracy	0.5600000
Precision	0.4705882
Recall	0.8000000
F1 Score	0.5925926
4 rows	

Based on these results, the accuracy of the model is 0.56, with a precision of 0.47, a recall of 0.80, and an F1 score of 0.59. Overall, the model performs reasonably well in detecting Manchester United images (high recall) but makes a notable number of incorrect predictions, resulting in lower precision.

```
df_train <- data.frame(X_train)
df_train$label <- y_train

kfold <- 5
set.seed(123)
fold_id_glm <- sample(rep(1:Kfold, length.out = length(y_train)))
f1_glm <- sapply(1:Kfold, function(fold) {
  val <- fold_id_glm == fold
  tr <- !val

  glm_fit <- glm(label ~ ., data = df_train[tr, ], family = binomial)

  prob_val <- predict(glm_fit, newdata = df_train[val, ], type = "response")
  pred_val <- ifelse(prob_val >= 0.5, "ManUtd", "Chelsea")
  pred_val <- factor(pred_val, levels = levels(y_train))

  metrics_binary(y_train[val], pred_val, positive = "ManUtd")$f1
})
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
mean_f1_glm_cv <- mean(f1_glm, na.rm = TRUE)
mean_f1_glm_cv
```

```
## [1] 0.5410909
```

To evaluate the performance of the logistic regression model, I used 5-fold cross-validation on the training set. In each fold, the model was trained on four-fifths of the data and evaluated on the remaining one-fifth using the F1 score for the ManUtd class. The vector `f1_glm` stores the F1 scores from all five folds, and their average is computed as `mean_f1_glm_cv`. The resulting mean F1 score is 0.54, which summarizes the overall cross-validated performance of the logistic regression model on the training data.

```
glm_fit_full <- glm(label ~ ., data = df_train, family = binomial)
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
df_test <- data.frame(X_test)
prob_test_glm <- predict(glm_fit_full, newdata = df_test, type = "response")

pred_test_glm <- ifelse(prob_test_glm >= 0.5, "ManUtd", "Chelsea")
pred_test_glm <- factor(pred_test_glm, levels = levels(y_train))

metrics_glm_test <- metrics_binary(y_test, pred_test_glm, positive = "ManUtd")

res_glm <- metrics_glm_test

glm_results_table <- data.frame(
  Metric = c("Accuracy", "Precision", "Recall", "F1 Score"),
  Value = c(res_glm$accuracy, res_glm$precision, res_glm$recall, res_glm$f1)
)

res_glm$confusion
```

```
##           y_pred
## y_true   Chelsea ManUtd
## Chelsea      6      9
## ManUtd      4      6
```

A logistic regression model was trained on the PCA features of the training set and then evaluated on the test set. The confusion matrix shows that 6 Chelsea images were correctly classified, while 9 Chelsea images were incorrectly predicted as ManUtd. For the ManUtd class, 6 images were correctly classified and 4 were misclassified as Chelsea. This indicates that the logistic regression model has moderate performance, with a noticeable number of misclassifications for both classes, and does not clearly outperform the KNN model in this setting.

```
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.5.2
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```

positive_class <- "ManUtd"
y_train_bin <- ifelse(y_train == positive_class, 1, 0)

X_train_mat <- as.matrix(X_train)
X_test_mat <- as.matrix(X_test)

fit_glmnet_model <- function(alpha_value) {
  set.seed(123)
  cv_fit <- cv.glmnet(
    x = X_train_mat,
    y = y_train_bin,
    family = "binomial",
    alpha = alpha_value,
    nfolds = 5
  )

  prob_test <- predict(cv_fit, newx = X_test_mat, s = "lambda.min", type = "response")
  pred_test <- ifelse(prob_test >= 0.5, positive_class, "Chelsea")
  pred_test <- factor(pred_test, levels = levels(y_train))

  metrics_binary(y_test, pred_test, positive = positive_class)
}

metrics_l1_test <- fit_glmnet_model(alpha_value = 1)

metrics_l2_test <- fit_glmnet_model(alpha_value = 0)

l1_l2_results <- data.frame(
  Model = c("Logistic L1 (Lasso)", "Logistic L2 (Ridge)"),
  Accuracy = c(metrics_l1_test$accuracy,
               metrics_l2_test$accuracy)
)

l1_l2_results

```

Model <chr>	Accuracy <dbl>
Logistic L1 (Lasso)	0.68
Logistic L2 (Ridge)	0.56
2 rows	

The L1 (Lasso) and L2 (Ridge) logistic regression models were trained using 5-fold cross-validation with glmnet. After selecting the optimal value of λ , both models were evaluated on the test set. The results show that the L1 model achieves an accuracy of 0.68, whereas the L2 model achieves an accuracy of 0.56. This indicates that the L1-regularized model performs better in this dataset, likely because it selects more informative features and reduces noise introduced by irrelevant PCA components.

```

metrics_to_row <- function(m, name) {
  data.frame(
    Model      = name,
    Accuracy   = m$accuracy,
    Precision  = m$precision,
    Recall     = m$recall,
    F1         = m$f1
  )
}

results_table <- rbind(
  metrics_to_row(metrics_knn_test, "KNN"),
  metrics_to_row(metrics_glm_test, "Logistic (no reg)"),
  metrics_to_row(metrics_l1_test, "Logistic L1"),
  metrics_to_row(metrics_l2_test, "Logistic L2")
)

results_table

```

Model <chr>	Accuracy <dbl>	Precision <dbl>	Recall <dbl>	F1 <dbl>
KNN	0.56	0.4705882	0.8	0.5925926
Logistic (no reg)	0.48	0.4000000	0.6	0.4800000
Logistic L1	0.68	0.5714286	0.8	0.6666667
Logistic L2	0.56	0.4666667	0.7	0.5600000

4 rows

To compare the different classification methods, I collected the evaluation metrics for all four models (KNN, logistic regression without regularization, logistic regression with L1 regularization, and logistic regression with L2 regularization) into a single results table. The helper function `metrics_to_row()` takes the output of the `metrics_binary()` function for a model and its name, and returns a one-row data frame containing the model name and its Accuracy, Precision, Recall, and F1 score. These rows are then combined with `rbind()` to form `results_table`, which summarizes the performance of all models.

The table shows that the Logistic L1 model achieves the best overall performance, with an accuracy of 0.68 and an F1 score of 0.67, outperforming KNN, logistic regression without regularization, and Logistic L2. KNN and Logistic L2 have similar performance (accuracy around 0.56), while logistic regression without regularization performs the worst with an accuracy of 0.48. Overall, these results suggest that adding L1 regularization to logistic regression improves the model's ability to generalize on this dataset, likely by selecting more informative PCA-based features and reducing overfitting.

Part 3

```

make_nonlinear_features <- function(Z, num_interact = 5){
  Z <- as.data.frame(Z)
  p <- ncol(Z)
  names(Z) <- paste0("PC", seq_len(p))

  base <- Z

  sq <- base^2
  names(sq) <- paste0(names(base), "_sq")

  lg <- log(abs(base) + 1)
  names(lg) <- paste0(names(base), "_log")

  m <- min(num_interact, p)
  pairs <- combn(m, 2)
  inter<- sapply(seq_len(ncol(pairs)), function(k){
    i <- pairs[1,k]
    j <- pairs[2,k]
    base[[i]] * base[[j]]
  })
  inter <- as.data.frame(inter)
  names(inter) <- paste0("PC", pairs[1, ], "_x_PC", pairs[2, ])

  cbind(base, sq, lg, inter)
}

```

The function `make_nonlinear_features()` expands the PCA feature set by creating several nonlinear transformations. It starts with the original PCA components, then adds their squared values, log-transformed versions ($\log(|PC| + 1)$), and interaction terms between the first few principal components. These additional features allow the classifiers to capture more complex patterns than with PCA features alone. The function returns a single data frame containing all original and nonlinear features.

```

Z_nl_all <- make_nonlinear_features(Z_all)

Xnl_train <- Z_nl_all[train_idx, ]
Xnl_test  <- Z_nl_all[-train_idx, ]

y_train <- labels[train_idx]
y_test  <- labels[-train_idx]

```

The nonlinear features are first created from the PCA components, producing an expanded feature set that includes squared terms, log transformations, and interaction features. Using the same train and test indices as before, this expanded dataset is then divided into a training set and a test set. The class labels are split in the same way. This ensures that the models in Part 3 are evaluated on the same images as in Part 2, but now using a richer nonlinear feature space.

```

library(class)

Kfold <- 5
k_grid <- c(1, 3, 5, 7, 9, 11)

set.seed(123)
fold_id <- sample(rep(1:Kfold, length.out = length(y_train)))

cv_f1_knn_n1 <- sapply(k_grid, function(k) {
  f1_per_fold <- sapply(1:Kfold, function(fold) {
    val <- fold_id == fold
    tr <- !val

    pred_val <- knn(
      train = Xnl_train[tr, , drop = FALSE],
      test  = Xnl_train[val, , drop = FALSE],
      cl    = y_train[tr],
      k     = k
    )

    metrics_binary(y_train[val], pred_val, positive = "ManUtd")$f1
  })

  mean(f1_per_fold, na.rm = TRUE)
})

best_k_n1 <- k_grid[which.max(cv_f1_knn_n1)]
best_k_n1

```

```
## [1] 7
```

This code uses 5-fold cross-validation to choose the best number of neighbours (k) for the KNN classifier when using the nonlinear feature set. For each value in k_grid (1, 3, 5, 7, 9, 11), the model is trained and evaluated on the training data, and the mean F1-score across the 5 folds is computed. Last line selects the k with the highest average F1-score. The output means that k = 7 gives the best cross-validation performance for KNN with the nonlinear features.

```

knn_n1_test_pred <- knn(
  train = Xnl_train,
  test  = Xnl_test,
  cl    = y_train,
  k     = best_k_n1
)

metrics_knn_n1_test <- metrics_binary(y_test, knn_n1_test_pred, positive = "ManUtd")

```



```
df_train_nl <- data.frame(Xnl_train, label = y_train)

Kfold <- 5
set.seed(123)
fold_id <- sample(rep(1:Kfold, length.out = nrow(df_train_nl)))

f1_glm_nl <- sapply(1:Kfold, function(k) {
  val_idx <- fold_id == k
  tr_idx <- !val_idx

  glm_fit <- glm(label ~ ., data = df_train_nl[tr_idx, ], family = binomial)

  prob_val <- predict(glm_fit, newdata = df_train_nl[val_idx, ], type = "response")
  pred_val <- factor(
    ifelse(prob_val >= 0.5, "ManUtd", "Chelsea"),
    levels = levels(y_train)
  )

  metrics_binary(y_train[val_idx], pred_val, positive = "ManUtd")$f1
})
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
```

```
mean_f1_glm_nl_cv <- mean(f1_glm_nl, na.rm = TRUE)
mean_f1_glm_nl_cv
```

```
## [1] 0.6022222
```

This code evaluates a logistic regression model trained on the nonlinear feature set using 5-fold cross-validation. In each fold, the model is trained on 80% of the training data and tested on the remaining 20%. The F1-score is computed for each fold, and these values are averaged to measure the model's overall performance.

The final output shows that the mean cross-validated F1-score is 0.60, indicating that logistic regression performs better with nonlinear features than with PCA features alone.

```
glm_nl_fit_full <- glm(label ~ ., data = df_train_nl, family = binomial)

df_test_nl <- data.frame(Xnl_test)
prob_test_glm_nl <- predict(glm_nl_fit_full, newdata = df_test_nl, type = "response")
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases
```

```
pred_test_glm_nl <- factor(
  ifelse(prob_test_glm_nl >= 0.5, "ManUtd", "Chelsea"),
  levels = levels(y_train)
)

metrics_glm_nl_test <- metrics_binary(y_test, pred_test_glm_nl, positive = "ManUtd")
metrics_glm_nl_test
```

```
## $precision
## [1] 0.3571429
##
## $recall
## [1] 0.5
##
## $f1
## [1] 0.4166667
##
## $accuracy
## [1] 0.44
##
## $confusion
##           y_pred
## y_true    Chelsea ManUtd
## Chelsea      6      9
## ManUtd      5      5
```

The logistic regression model trained with the nonlinear feature set was evaluated on the test data. The confusion matrix shows that the model correctly classified six Chelsea images and five Manchester United images, while misclassifying nine Chelsea and five ManUtd samples. Based on these predictions, the model achieved an accuracy of 0.44, a precision of 0.36, a recall of 0.50, and an F1 score of 0.42. These results indicate that, even with nonlinear features, logistic regression performs relatively poorly and does not capture the class structure as effectively as the other nonlinear models.

```

library(glmnet)

positive_class <- "ManUtd"
y_train_bin <- ifelse(y_train == positive_class, 1, 0)

Xnl_train_mat <- as.matrix(Xnl_train)
Xnl_test_mat  <- as.matrix(Xnl_test)

fit_glmnet_nl <- function(alpha_value) {
  set.seed(123)
  cv_fit <- cv.glmnet(
    x = Xnl_train_mat,
    y = y_train_bin,
    family = "binomial",
    alpha = alpha_value,
    nfolds = 5
  )

  prob_test <- predict(cv_fit, newx = Xnl_test_mat, s = "lambda.min", type = "response")
  pred_test <- ifelse(prob_test >= 0.5, positive_class, "Chelsea")
  pred_test <- factor(pred_test, levels = levels(y_train))

  metrics_binary(y_test, pred_test, positive = positive_class)
}

metrics_l1_nl_test <- fit_glmnet_nl(alpha_value = 1)
metrics_l2_nl_test <- fit_glmnet_nl(alpha_value = 0)

```

This code fits L1 (Lasso) and L2 (Ridge) logistic regression models using the nonlinear feature set. It uses 5-fold cross-validation to choose the best value of λ , then predicts the test labels and evaluates performance with accuracy, precision, recall, and F1 score. The results stored in `metrics_l1_nl_test` and `metrics_l2_nl_test` show how each regularized model performs with the expanded nonlinear features.

```

results_table_nl <- rbind(
  metrics_to_row(metrics_knn_nl_test, "KNN (nonlinear)"),
  metrics_to_row(metrics_glm_nl_test, "Logistic (nonlinear)"),
  metrics_to_row(metrics_l1_nl_test, "Logistic L1 (nonlinear)"),
  metrics_to_row(metrics_l2_nl_test, "Logistic L2 (nonlinear)")
)

results_table_nl

```

Model <chr>	Accuracy <dbl>	Precision <dbl>	Recall <dbl>	F1 <dbl>
KNN (nonlinear)	0.40	0.3684211	0.7	0.4827586
Logistic (nonlinear)	0.44	0.3571429	0.5	0.4166667
Logistic L1 (nonlinear)	0.56	0.4666667	0.7	0.5600000
Logistic L2 (nonlinear)	0.64	1.0000000	0.1	0.1818182

4 rows

The nonlinear feature results show that only the regularized logistic models benefit meaningfully from the expanded feature set. Logistic L1 improves compared to the PCA-only version, reaching higher accuracy and a better-balanced F1 score. Logistic L2 achieves the highest accuracy, though its very low recall limits its usefulness. In contrast, KNN and standard logistic regression perform worse than in the PCA-only setting. Overall, nonlinear transformations help some models—mainly the regularized logistic ones—while offering little or no improvement for the others.

Part 4 – Final Model Selection & Error Analysis

Across all models evaluated in Parts 2 and 3, the logistic regression model with L1 regularization (Lasso) trained on the first 76 PCA components is selected as the best overall classifier.

For the PCA-based models, the Logistic L1 model achieves:

- Accuracy ≈ 0.68
- Precision ≈ 0.57
- Recall ≈ 0.80
- F1-score ≈ 0.67

This makes it the only model that simultaneously maintains relatively high accuracy, balanced precision–recall, and the highest F1-score. KNN and unregularized logistic regression clearly underperform compared to this model, and the Logistic L2 model on PCA features does not provide any consistent improvement.

Although some nonlinear models (based on the expanded feature set) reach competitive or slightly higher accuracy in certain cases, they either sacrifice recall dramatically or suffer from worse overall F1 scores. In particular, the nonlinear Logistic L2 model attains a higher accuracy but an extremely low recall and F1 score, meaning that it effectively fails to detect one of the classes. Therefore, the Logistic L1 (Lasso) model using 76 PCA components is selected as the final and best-performing classifier.

Below, I perform a more detailed error analysis of this model by examining its false positive (FP) and false negative (FN) predictions on the held-out test set, and by interpreting visual patterns that may explain these misclassifications.

```
library(glmnet)

set.seed(123)
cv_l1_pca <- cv.glmnet(
  x      = X_train_mat,
  y      = y_train_bin,
  family = "binomial",
  alpha  = 1,
  nfolds = 5
)

prob_test_l1 <- predict(
  cv_l1_pca,
  newx = X_test_mat,
  s     = "lambda.min",
  type = "response"
)[, 1]

pred_test_l1_chr <- ifelse(prob_test_l1 >= 0.5, positive_class, "Chelsea")
pred_test_l1 <- factor(pred_test_l1_chr, levels = levels(y_test))

metrics_l1_test <- metrics_binary(y_test, pred_test_l1, positive = positive_class)

metrics_l1_test$confusion
```

```
##          y_pred
## y_true   Chelsea ManUtd
## Chelsea      9      6
## ManUtd       2      8
```

```
metrics_l1_test
```

```
## $precision
## [1] 0.5714286
##
## $recall
## [1] 0.8
##
## $f1
## [1] 0.6666667
##
## $accuracy
## [1] 0.68
##
## $confusion
##           y_pred
## y_true   Chelsea ManUtd
## Chelsea      9      6
## ManUtd       2      8
```

```
is_fp_test <- (pred_test_l1 == "ManUtd") & (y_test == "Chelsea")

is_fn_test <- (pred_test_l1 == "Chelsea") & (y_test == "ManUtd")

fp_test_idx <- which(is_fp_test)
fn_test_idx <- which(is_fn_test)

length(fp_test_idx)
```

```
## [1] 6
```

```
length(fn_test_idx)
```

```
## [1] 2
```

```
test_ids <- setdiff(seq_len(nrow(x)), train_idx)

orig_fp_idx <- test_ids[fp_test_idx]
orig_fn_idx <- test_ids[fn_test_idx]

orig_fp_idx
```

```
## [1] 10 11 18 33 56 62
```

```
orig_fn_idx
```

```
## [1] 113 116
```

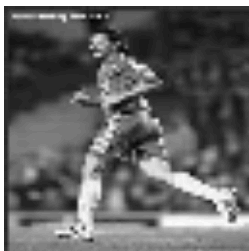
```
show_n_fp <- min(3, length(orig_fp_idx))
show_n_fn <- min(3, length(orig_fn_idx))

par(mfrow = c(2, max(show_n_fp, show_n_fn)))

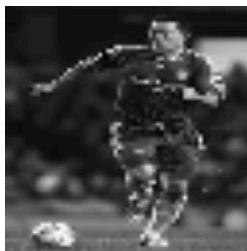
for (i in seq_len(show_n_fp)) {
  idx <- orig_fp_idx[i]
  true_lab <- labels[idx]
  pred_lab <- "ManUtd"
  show_image(
    x[idx, ],
    title = paste0(
      "False Positive\n",
      "True: ", true_lab, " Pred: ", pred_lab,
      "\nImage index: ", idx
    )
  )
}

for (i in seq_len(show_n_fn)) {
  idx <- orig_fn_idx[i]
  true_lab <- labels[idx]
  pred_lab <- "Chelsea"
  show_image(
    x[idx, ],
    title = paste0(
      "False Negative\n",
      "True: ", true_lab, " Pred: ", pred_lab,
      "\nImage index: ", idx
    )
  )
}
```

False Positive
True: Chelsea Pred: ManUtd
Image index: 10



False Positive
True: Chelsea Pred: ManUtd
Image index: 11



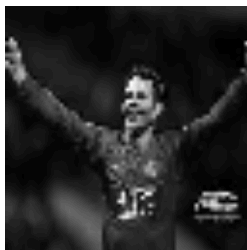
False Positive
True: Chelsea Pred: ManUtd
Image index: 18



False Negative
True: ManUtd Pred: Chelsea
Image index: 113



False Negative
True: ManUtd Pred: Chelsea
Image index: 116



Using the selected Logistic L1 model, I examined the predictions on the held-out test set and identified the false positive (FP) and false negative (FN) cases. False positives are images where the model predicts “Manchester United” but the true label is “Chelsea”, while false negatives are the opposite. Several representative FP and FN images are displayed in the figures above, with titles indicating the true and predicted labels.

In many false positives, the Chelsea players appear in poses or contexts that are visually similar to Manchester United images in the training set – for example, frontal full-body poses, similar background structure, or crowd patterns. Because the model operates on 64×64 grayscale images and PCA features, it does not have access to jersey color information and instead relies on coarse shape, edges, and textures. When these structural cues resemble those of typical Manchester United images, the model tends to misclassify Chelsea as Manchester United.

False negatives often occur in the opposite situation: Manchester United images in which the player is partially occluded, very small in the frame, or in an unusual pose. In these cases, the overall silhouette and local gradients deviate from the main patterns learned from the training data, and the model falls back to predicting the more “Chelsea-like” structures. Noisy or low-contrast images, as well as extreme lighting, also contribute to such mistakes.

Overall, the misclassifications reveal several limitations of the chosen model and feature pipeline:

1. **Limited PCA representation.** Although 76 principal components capture around 90% of the total variance, some of the left-out components may still contain discriminative information, especially for subtle differences between the two clubs' visual patterns. Reducing dimensionality too aggressively can blur out fine details that would help distinguish similar poses or backgrounds.

2. **Loss of color information.** All images are converted to grayscale before PCA. As a result, the model cannot use jersey colors (e.g., red vs. blue) or other color cues that are highly informative for distinguishing football teams. This forces the classifier to rely only on shape and texture, which are more ambiguous.
3. **Limited nonlinearity.** Logistic regression with L1 regularization is still a linear model in the PCA feature space. Even though some nonlinear transformations were explored in Part 3, the current approach cannot fully capture complex, high-order structures in the images, such as detailed logos, fine-grained textures, or subtle context cues.
4. **Data and resolution constraints.** The images are relatively low resolution (64×64) and the dataset contains only 124 examples (65 Chelsea and 59 Manchester United). This makes it difficult for a simple model to generalize to less common camera viewpoints, unusual poses, or heavily occluded players, which often appear among the FPs and FNs.

To reduce such errors, more advanced methods could be applied. Convolutional neural networks (CNNs) trained directly on higher-resolution RGB images would be able to exploit both local patterns and color information, and data augmentation could expose the model to a wider variety of poses and lighting conditions. Transfer learning from a pre-trained image classification network would likely give a large improvement over hand-crafted PCA features. In addition, using class-balanced loss functions or focusing on hard examples (e.g., via hard negative mining) could help the model allocate more capacity to the most ambiguous Chelsea vs. Manchester United cases.

In summary, while the Logistic L1 model on PCA features performs best among all models considered in this assignment, its false positives and false negatives highlight the inherent limitations of linear classifiers on compressed grayscale representations. More expressive models and richer input representations would be needed to further reduce these classification errors.